



Transcript

Week 20: Unsupervised Learning - Advanced

Video 1 - Unsupervised Learning - Advanced

How do machines uncover structure in chaos—without a single label to guide them?

This is where unsupervised learning steps in—teaching machines to spot patterns, find structure, and draw insights from raw data without any labels. It's like holding up a mirror to the data and letting it expose its own hidden truths. These advanced techniques go far beyond simple clustering—they help machines navigate and interpret complex, unstructured information. Among the most powerful tools are Autoencoders, which compress and rebuild data to reveal what's essential, often used in anomaly detection and dimensionality reduction. Then come Deep Belief Networks, multi-layered systems that learn deep features and even generate new data. And finally, Restricted Boltzmann Machines—compact models that laid the groundwork for many deep learning breakthroughs. Together, these methods form a core part of how machines learn to understand the world on their own.

Ready to see how machines learn to make sense of the unknown? Let's get started!

Video 2 - Introduction to Autoencoders

Autoencoders are a type of neural network built to learn from data—without needing labels. Their goal is to take input, compress it into a smaller version called the latent space, and then reconstruct it. This process happens in two stages. The encoder reduces the data, removing noise and redundancy. The decoder then tries to recreate the original input from that compressed version. Since the network is trained to rebuild what it started with, it learns to capture the most important patterns in the data—making it great for tasks like dimensionality reduction or anomaly detection.

Autoencoders help reduce the size of data while keeping its most important features. Think of them like a smart zip file. For example, when we feed in high-resolution images, the autoencoder compresses them by learning the key patterns. It then reconstructs the images from these patterns with little loss in quality. This technique is very useful for tasks such as reducing dimensions, finding unusual patterns, and removing noise from images. In the next part, we'll explore how each component of an autoencoder works step by step.

Now that we understand what autoencoders do, let's explore how they are structured. An autoencoder has five main components, each playing a unique role.

We begin with the **input layer**, where raw data such as images, text, or numerical values enters the model.

Next is the **encoder**, which compresses this data into a smaller, more efficient form. Think of it like summarising a high-resolution image using fewer details while keeping the most important patterns. The encoder uses several layers and mathematical operations to achieve



this compression.

The compressed data is passed to the **latent space**. This is the bottleneck in the network—where only the essential information is stored. It helps the model focus on what really matters and ignore irrelevant details.

Then, the **decoder** takes over. It takes the compact information from the latent space and tries to rebuild the original data as accurately as possible. This step essentially reverses the encoding process.

Finally, the **output layer** produces the reconstructed version of the original input.

To evaluate how accurate this reconstruction is, the model uses a **loss function** such as Mean Squared Error. This calculates the difference between the input and the output.

To help the model learn better, we also use **activation functions** like ReL—you—or Sigmoid. These add non-linearity, allowing the model to capture complex patterns in the data.

Altogether, this architecture allows autoencoders to perform powerful tasks like dimensionality reduction, denoising, and anomaly detection—especially in situations where labelled data is limited.

To better understand how autoencoders work, let's walk through a simple example—compressing an image.

Imagine you begin with a high-resolution image that is 256 by 256 pixels. This image enters the **input layer**. The **encoder** then processes this image and compresses it into a much smaller format—for instance, 32 by 32 pixels.

The result is stored in the **latent space**, which contains only the most essential details. It's like capturing the image's important features while ignoring everything unnecessary.

Next, the **decoder** uses this compressed version to reconstruct the image. It works to bring back as much detail as possible, though it may not be exactly identical to the original.

Finally, the **output layer** produces the reconstructed image. This version retains the main visual elements, even though it is based on the compressed data.

As shown in the flowchart on this slide, the process starts with a detailed input and moves through compression, representation, and reconstruction. Despite some differences in resolution, the reconstructed image still contains all the key information.

This example shows how autoencoders learn to compress data and then rebuild it effectively. This makes them useful for tasks like reducing image size, removing noise, or detecting unusual patterns in data.

Now let's see where autoencoders are actually used in the real world. One major application is **dimensionality reduction**. Autoencoders offer a smarter alternative to Principal Component Analysis, or P-C-A, especially when dealing with complex, non-linear data.

They're also helpful for **anomaly detection**. By learning what "normal" looks like, autoencoders can flag anything unusual—like fraudulent transactions in banking or



abnormalities in medical scans.

Another common use is **denoising**. Autoencoders can remove noise from images or audio—for example, cleaning up old photographs or distorted voice recordings.

They're also useful for **pretraining deep learning models**. In complex A-I tasks, autoencoders can be used to initialise model weights, helping speed up training and improve accuracy.

Here are some practical examples:

In **healthcare**, autoencoders can help detect issues in X-rays or scans faster.

In **finance**, they can spot unusual or suspicious activity.

And in **e-commerce**, they can recommend products by learning what users like.

These applications show how autoencoders can handle a wide range of challenges—from image clean-up to customer behaviour prediction. In the next session, we'll see how to build and use them ourselves.

Now that you've seen how autoencoders solve real-world problems, think about this: Where could you apply them in your field of study? Whether it's cleaning messy data, spotting patterns, or building smarter models—autoencoders give you the tools to make data work smarter. Ready to try building one yourself?

Video 3 - Introduction to Deep Belief Networks (DBNs)

Deep Belief Networks, or DBNs, are a powerful class of deep neural networks designed to learn from data without supervision. At their core, DBNs are built by stacking several layers of Restricted Boltzmann Machines or Autoencoders. Each of these layers acts like a building block—learning to represent features in the data more deeply than the one before it.

Training happens step by step. The first layer learns to detect simple patterns—like edges or textures. The next layer uses those to identify more abstract shapes, and the pattern continues. This gradual learning process allows DBNs to understand complex structures in data, even when no labels are provided.

Because of this deep, hierarchical structure, DBNs are used in a variety of tasks—such as extracting features for classification, generating realistic images or text, or simplifying data for visualization. Whether it's reading handwriting, compressing image data, or generating natural-sounding language, DBNs provide a layered way of thinking that mimi

To understand how Deep Belief Networks—or D-B-Ns—work, let's break down their structure.

First, we have the **input layer**, where raw data, like images or numbers, is fed into the network.

Next are the **hidden layers**, made up of stacked Restricted Boltzmann Machines—or R-B-Ms—or autoencoders. Each R-B-M learns a pattern in the data by building a probability model. The first layer handles the raw input, the second builds on that, and so on—layer by layer—forming a deeper understanding of the features.



Then comes the **output layer**. This can be used to generate new data or classify input, depending on whether the D-B-N is used for generation or for supervised learning.

Training a D-B-N has two parts: **pretraining** and **fine-tuning**. During pretraining, each R-B-M is trained independently in an unsupervised way using a method called **contrastive divergence**. After that, all the layers are combined and **fine-tuned** using labelled data if it's available.

This approach helps each layer learn meaningful patterns before building the full network. Let's look at an example of how this works in practice.

Now let's explore how Deep Belief Networks—spelled D-B-Ns—are used in real-world applications.

In **image recognition**, D-B-Ns can learn patterns from images to recognise digits, like those in the M-N-I-S-T dataset. By learning step by step, they can identify simple edges and combine them to form full shapes or numbers.

In **natural language processing**, or N-L-P, D-B-Ns can help understand text by learning word embeddings or sentence structures. This is useful for language-related tasks like translation or classification.

D-B-Ns also support **recommender systems**. They learn what users like and help suggest personalised content—like in e-commerce or streaming platforms.

Finally, in **data compression**, D-B-Ns reduce the number of features in large datasets while keeping the important ones. This helps make data easier to process without losing meaning.

These examples show how D-B-Ns apply their layered learning to solve real problems in many fields.

Before we move on to implementation, let's look at the strengths and limitations of Deep Belief Networks—or D-B-Ns.

One of the biggest strengths of D-B-Ns is their ability to learn **layered patterns** from complex data. This makes them great for tasks like feature extraction and dimensionality reduction. They can also handle **generative tasks**, such as creating new data, as well as **discriminative tasks**, like classification.

Another advantage is that D-B-Ns are useful for **unsupervised pretraining**. They help prepare deep networks for faster and more effective learning.

However, there are also some challenges. D-B-Ns can be **computationally expensive**, especially when training several Restricted Boltzmann Machines—spelled R-B-Ms. They also need careful **initialisation** and **hyperparameter tuning**, such as setting the right learning rates and number of hidden units.

Another issue is the **vanishing gradient problem**. In deeper networks, this can make it harder for the model to learn efficiently.

Thanks to improvements in computing power and newer hybrid models, many of these challenges can be managed. With this understanding, we're now ready to move forward and take a closer look at the building blocks of D-B-Ns—starting with R-B-Ms.



Now that you've explored the strengths and limitations of Deep Belief Networks, take a moment to reflect—how would you decide if a D-B-N is the right choice for your problem? Think about the trade-offs between accuracy and computational cost. As we move ahead, consider where you might apply this knowledge in real research or projects.

Video 4 - Introduction to Restricted Boltzmann Machines

Restricted Boltzmann Machines, or RBMs, are a type of neural network that learns to find hidden patterns in data—by modeling how likely certain features are to appear together. They're made of two layers: a visible layer that takes in the input, and a hidden layer that discovers the deeper features. These layers are connected with weights and have their own biases.

What sets RBMs apart is their use of an energy function. The network learns by minimizing this energy, which helps it figure out which combinations of inputs and features are most meaningful. This makes RBMs great for feature learning, dimensionality reduction, and even recommender systems. They're also used as a stepping stone to train deeper models, like Deep Belief Networks.

Let's explore how a Restricted Boltzmann Machine—or R-B-M—works.

An R-B-M has two layers. The **visible layer**, written as v , holds the observed input data, such as images or numerical values. The **hidden layer**, written as h , learns useful patterns that are not directly visible in the input.

These two layers are connected using a **weight matrix**, shown as W . Each visible and hidden unit also has a **bias term**— b for visible units and c for hidden ones.

The network learns by minimising an **energy function**, which is written as:

E of v and h equals negative summation of b_i times v_i , minus summation of c_j times h_j , minus summation of v_i times h_j times W_{ij} .

By reducing this energy, the model improves its understanding of the relationships between the visible and hidden layers. This structure—combined with restricted connectivity—makes R-B-Ms both efficient and effective for unsupervised learning.

To train a Restricted Boltzmann Machine—or R-B-M—we start by **initialising** the weights and biases with small random values.

The training begins with a **forward pass**. Here, we calculate the probability that each hidden unit is activated, based on the current visible units. This is done using a sigmoid function. The model takes the weighted sum of the visible units, adds the bias for the hidden unit, and applies the sigmoid function to produce a probability between zero and one.

Next is the **reconstruction phase**, where the model tries to rebuild the visible units from the hidden layer. It applies the same steps in reverse—this time calculating the probability of each visible unit being active, based on the hidden units, again using the sigmoid function.

Then comes **Contrastive Divergence**, the learning step. The model compares the joint



activation of visible and hidden units from two moments:

first, during the forward pass—called the **positive phase**—and then during the reconstruction phase—called the **negative phase**.

The weights are updated by subtracting the negative phase result from the positive one, and scaling the difference using the **learning rate**. This helps the model reduce errors in reconstruction and learn important patterns in the data.

By repeating this process—forward pass, reconstruction, and weight update—the R-B-M gradually becomes better at representing the training data. This makes it useful for tasks like feature extraction and collaborative filtering.

Restricted Boltzmann Machines—spelled R-B-Ms—are used in several real-world applications thanks to their ability to learn hidden patterns in data.

In **collaborative filtering**, R-B-Ms are used in systems like Netflix to recommend content. They learn user preferences by analysing interactions between users and items.

For **feature learning**, R-B-Ms are great at finding hidden features in data, which helps reduce dimensionality or improve supervised models.

In **generative modelling**, they can create new data samples—such as synthetic images or text—by sampling from learned probability distributions.

And for **pretraining deep networks**, R-B-Ms are used to initialise weights in models like D-B-Ns, helping speed up training and improve results.

These versatile applications make R-B-Ms a strong foundation in many unsupervised learning pipelines.

Can you think of a platform you use that might rely on hidden patterns to personalise your experience? As we've seen, Restricted Boltzmann Machines power everything from recommendations to feature extraction. Now it's your turn to explore how these concepts shape the tools and technologies around us.

Video 5: Advanced Unsupervised Learning Part 1- Demo

Hello and welcome everyone. In this session, we are going to learn about advanced unsupervised learning technique. And the algorithm we are going to see here today is about the autoencoders.

You might have seen that in the slides that autoencoder is a type of neural network, which is specially designed to compress or to learn the compressed representation of your data. And since we do not require a target label over here, we would assume that this is good for unsupervised tasks. And you would see that this autoencoder model is very beneficial for tasks where we would want to extract the compressed set of features.

And that's why it's very useful in dimensionality reduction in learning those essential features



of your data. And at the same time, because it's extracting those important features, we can use this to denoise the data. You can see on your screen, the picture over here where you have the input layer, which consumes the input.

It may be in the form of an image or any kind of data. And then this is projected onto a latent space. This latent space is usually less than the size of your input layer.

And this latent space acts as a bottleneck, that is, any information about this input. If it's trying to go on the other side, it has to go through this layer. Now, once we have the information passed through this bottleneck layer, we then project it onto the decoder.

The decoder's job is nothing but to reconstruct the input in the form of an output. So whatever input you're trying to pass over here, you're going to expect the decoder to generate the same kind of output over here. Now we are going to model this network and train this network to do so.

And then see that how efficient it is with different sizes of latent space. Let's proceed ahead. We have the description over here where we are talking about the encoder compressing the input data into a lower dimension representation, and we call that as a latent space.

And the decoder's job is to reconstruct the original data from the compressed latent representation, which we have just created above. Well, let's jump to the part where you would want to start with. So the first thing is to import the libraries which are required.

Now we are going to use NumPy for basic data handling. We are going to use TensorFlow here and we are going to use the layers within TensorFlow, which acts as building blocks like the dense layers for our auto-encoder network. Let's start with the first command over here where we are loading this MNIST dataset.

If you are aware of this MNIST dataset, it's a handed digit dataset. So it looks something like this. This is one of our those legacy dataset, which has been very commonly used for learning.

Now this is a 28 cross 28 greyscale image of handwritten digits you can see. So we are going to use this dataset for our learning and we load that dataset. Since this dataset is very common, it's actually available within Keras itself.

And I don't need the test labels because, you know, we are only going to live with your trained data. Let's try to normalise them. So we are going to normalise the pixels.

You know that each pixel ranges from 0 to 255. So I'm going to normalise that. Usually, you know that normalisation is done to achieve a faster and efficient training.

After we have normalised that, we are going to flatten that because we are going to send that to our neural network, which is going to receive a flat input. So we are going to flatten each image into a single 784-dimension vector. Why is it 784? Because 28 cross 28 would make it 784, right? So 28 cross 28 image will make it 784.

Well, now we are going to decide the size of the latent space. This is very important. Here, we are going to decide the size.



And as I said that my input is 784, right? 28 cross 28 would be 784. And I need to project that 784 to some dimension lower than 784. And that's where we have decided that we'll do that on encoding dimension of 32.

So we are going to compress the 784 input values into 32 values, right? How do we do that? Let's see. So that's what we are going to do by building the encoder part. If you see here, the first step here is the input layer.

Now, this is going to accept a shape of 784. That's a vector of size 784. Then there is a first dense layer, which is going to take this 784 and reduce the size to 128 neurones.

And while it does so, we are going to apply an activation of value. The next layer then takes this 128-sized vector and reduces it to 64 for the reduction. And then this 64 is then taken and again reduced to your 32.

You see how a gradual reduction 784 to 128 to 64 or 32. You can actually play with this. You can change these numbers as per your wish.

And you can try to see its effect on the output. That's your encoder. Now my image has been projected down to that latent space representation.

Now we go by building the decoder. Where we take the encoded image. And now I try to mirror the encoder's structure in the reverse order.

So I took 64 and projected it to 32, right? Now from 32, I'll go back to 64. So I came to 64 from 128-sized vector. So I'll go from 64 to 128.

And 128 was achieved from 74-sized vector. So I'll project it back from 128 to 784. So in a way, I'm expanding it and trying to reconstruct the original 784-length vector.

Now while I do so, you have to be very careful about the activation function at the last layer. We are going to use sigmoid. Remember, when I had normalised my data, right? So the range is between 0 to 1. So every pixel has to be between 0 and 1. And that's the property of your sigmoid activation function.

That it will provide you the output in the range of 0 to 1. Well, we have built our encoder and decoder separately. Let's try to connect them. So I can create a Keras model where I am going to pass the input image and the decoder.

The input will go to the encoder, get compressed, and the decoder will then reconstruct the output. This is my complete autoencoder network. Well, we are going to compile the autoencoder.

Now you know that while we compile, we have to set the optimiser. I'm going to use the most widely used Adam optimiser for faster training. And we are going to use a binary cross entropy loss.

Because we are predicting probabilities and they are between 0 and 1. Now we go by fitting the model on X-train and X-train. This is very important. This is something which is unique to autoencoders.



So the input and output, they are actually the same. And this is the reason, as I said, that the autoencoder learns to reconstruct its own input. Now we are going to do that for 20 epochs with a batch size of 256.

Again, this is something which you can change. Each of these values you can change. Epochs actually is nothing but how many times you want the training data to run across your network.

You can increase that further. And even batch size, depending upon your RAM, you can increase the batch size. I shuffle my data, and I use my test data for validation, whether it's able to reconstruct the output or not.

Now, once the training is done, you will see that the loss, you see that the loss is reducing, reducing and reducing, right? Now, after 20 epochs, we are going to then ask the encoder, the autoencoder to actually reconstruct the image on X-test. Now, while I do so, the autoencoder will compress and reconstruct the image. And that's how we have the reconstructed images stored in this encoded images.

Now, I want to go ahead, and I want to visualise these original and reconstructed images. And this is a code to actually visualise. So I'm saying that I want to display 10 images.

So if you see here, we have 10 images. And I want to display the original and the reconstructed in that order. So this is my original and these are my reconstructed.

So if you see, my original and reconstructed images are almost similar. So you can observe that the autoencoder has learned to capture those essential features of each digit. And we see a good reconstruction, means that the network has successfully learned the efficient representation of the data in the latent space.

That completes this notebook. And I would request you to play around with few of these values. For example, playing with the encoding dimension, increasing the number of layers over here, and increasing the number of layers accordingly over here.

Playing with these number of neurones, both in the encoder and decoder. And then going for a larger epoch set, a different batch size. These are some hyperparameters you can try.

And that would give you slightly different result in terms of the numbers. But yes, in terms of the output, you won't tend to see anything better than what we have on our screen. Thank you.

Video 6: Advanced Unsupervised Learning Part 2- Demo

Welcome back everyone. In this session we are going to see another type of unsupervised learning neural network we have previously seen, not wind coders. We are going to see this deep belief network.

Because we are going to demonstrate this concept by stacking multiple dense layers. In a layer wise manner, similar to how we have done in auto encoders. Now a deep belief network is a kind of unsupervised neural network.



And it is actually made by stacking multiple layers of restricted Boltzmann machines. Where each layer is learning to represent features of the input in a hierarchical manner. The lower layers, they learn those simple patterns.

While the deeper layers, they would learn more complex and abstract features. Now you would observe that in a deep belief network. The RBMs are trained one at a time in a greedy manner.

And after this pre-training of RBMs, the entire network can be fine-tuned using supervised learning. In Keras, we are going to demonstrate that by stacking these dense layers and training them progressively. So you can see that there would be an input layer.

There is an input layer where you are going to feed your input. And this actually consumes the raw input features. Once we have the input layer, it then passes to the hidden layers.

Which are stacked layers, one over the other. And each of these hidden layers, they are acting like RBMs. And finally you have an output layer, which is a classifier.

So each hidden layer would learn to extract those abstract features. And in an actual deep belief network, you would see that each layer weights are learned in an unsupervised manner first. And then the whole stack is then fine-tuned for some downstream task.

Let's try to see that. In a similar fashion, we are going to import those essential libraries. The NumPy, the Keras library to build a network.

And I'm going to load the same MNIST dataset, which we have done in the previous demonstration. Now we are going to keep the labels this time. Because our deep belief network will eventually be fine-tuned as a classifier.

Now as usual, we are going to normalise our dataset between 0 to 1. Where it was originally 255 pixels, but when we divide that by 255, it would get normalised. And then we are going to flatten it. There is one more operation on the label data.

That is we convert that to one hot encoded vector. This is basically conversion of the labels into one hot encoded for multi-class classification. Now in a real deep belief network, we would be attempting on an RBM, which is trained in an unsupervised manner.

So in this example, we simulate it with a dense layer. This is that dense layer, which is going to do what? It's going to take my 784-size vector and compress that into 512-dimension representation. And then the second layer takes the output of the first hidden layer, that is this one.

And learns more abstract features by converting 512 to 226. So this is my first RBM and this is my second RBM layer. And each of these, this is learning some simple patterns, this is learning some complex patterns.

Now the third layer would deepen this hierarchy of learned representations. And that would go as deeper RBMs do in a true deep belief network. Once we are satisfied by the number of layers and the kind of representation it could learn, we finish it by adding an output layer.



Now here we are going to have 10 units, because we have 10 digits and 1 unit for each of these digits. And you understand that the network's goal is to map the learned features over here to one of these digit predictions. Now I'm going to collect all these and create a single model.

We're going to wrap these layers into a single model. And this is something which represents my deep belief network structure. We go by compiling this structure using our atom optimiser.

And because I'm using 10 as an output, so I'm going to use categorical cross entropy. And I'm going to use accuracy as a metric to evaluate how good my model is doing. Now we train the deep belief network end to end, where we're going to pass the X train and Y train.

In a real deep belief network, each layer would be pre-trained and supervised and then fine-tuned. In this example, we are actually stacking the entire network and training it directly. And that's more a practical way in modern frameworks.

You would see that it's good to train that. And you can see the accuracy is moving on the higher side. And after 20 epochs, we have around a 99% accuracy and a 97% accuracy on validation.

Now we can evaluate the performance on test data. I see that there is a 97.52% accuracy. So we know that autoencoders are versatile tools for unsupervised learning.

And their main strength is learning how to compress data into a lower dimensional representation. They can be useful for many tasks like visualisation or reducing computational load. But with nonlinear capabilities, they can actually uncover more complex patterns.

But there are certain limitations to what autoencoders can solve. We try to incorporate that using these deep belief networks. And the big idea here is to learn good feature representation in an unsupervised manner.

Way before I fine-tune for a supervised task. So there is a layer-wise pre-training. Which is especially important before modern activation functions.

And that's where DBNs shine. When you have a lot of unlabelled data and only a small amount of labelled data. Because they can learn the pattern from the unlabelled data first.

And then use those learned features to improve performance in the labelled task. Besides classification, DBNs can also reduce dimensionality. Very similar to autoencoders.

By learning these hierarchical features, they transform the raw data into compressed, abstract representations. And they are the ones which are easier to work with. There is another classic use case which is transfer learning.

So the pre-trained layers from a deep belief network can be used to initialise other deep models. And that's what saves the training time. Improves the stability.



Especially when you are training really deep architectures. While we see that modern learning has moved on to architectures like CNNs and RNNs and transformers. Autoencoders and DBNs remain important foundations for understanding how deep neural networks learn to represent complex data.

Thank you everyone.